Bats Mitogenome Project

Milana Serkin

2025-12-12

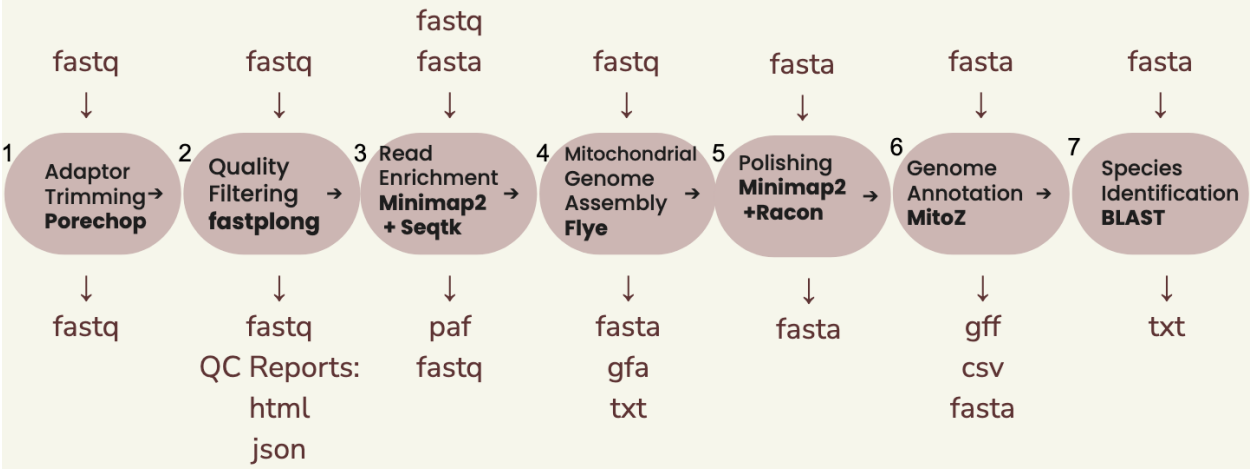
The dataset was generated from a bat spleen collected in Vietnam. DNA was extracted from the spleen and prepared with sequencing adapters for Oxford Nanopore sequencing. The raw sequencing data were exported as FASTQ files. The goal is to identify the bat species using its mitochondrial DNA. To do this, the dataset will be trimmed, filtered, and assembled into a mitochondrial genome, which will then be analyzed for species identification.

Key question prompts:

1. Briefly describe the biological question and why mitochondrial genomes are useful for species identification. The biological question is: Which bat species does this dataset come from? Mitochondrial genomes help answer this question because the protein-coding genes in mitochondrial DNA act as a "barcode" for species. Mitochondrial DNA mutates faster than nuclear DNA and is highly conserved within species, making it useful for species identification.
2. What are the main data inputs (e.g., FASTQ files) and expected outputs of the pipeline? The main data input is the raw FASTQ file containing Oxford Nanopore sequencing reads extracted from the bat spleen. The expected output is a FASTA file of the complete mitochondrial genome assembly. This assembled genome can then be used in a search tool (like BLAST) to identify the species.

Part 2: Pipeline Diagram

```
knitr::include_graphics("pipeline.png")
```



Narrative questions:

1. In 4–6 sentences, walk through the pipeline shown in your diagram, explaining the flow from raw reads to species ID.
2. For each step, specify the main tool/software and the file format in and out (e.g., FASTQ in → trimmed FASTQ out).

The adapters on the raw reads (FASTQ) are trimmed using Porechop, producing a trimmed FASTQ. The trimmed FASTQ is then filtered with FastPlong, which removes very short or low-quality reads, giving a cleaned FASTQ along with HTML and JSON quality reports. The cleaned FASTQ is mapped using Minimap2 to produce a PAF file, which contains the alignment information used for downstream assembly. The reads and alignment info are then processed in Flye to produce multiple FASTA mitochondrial genome assemblies. The best genome assembly is chosen and the FASTA file is polished using Racon, generating the final FASTA mitochondrial genome. This polished genome(FASTA) is annotated with MitoZ, producing GenBank and GFF files of gene coordinates, and finally compared with known sequences using BLAST to identify the closest bat species(TXT).

Part 3: Adapter Trimming with Porechop

#SLURM script for Porechop

```
#!/bin/bash
#SBATCH --job-name=porechop_bat
#SBATCH --output=porechop_%j.out
#SBATCH --error=porechop_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=100G
```

```
#SBATCH --time=9:00:00

# RUN PORECHOP
~/Porechop/porechop-runner.py \
    -i 00_rawReads_bat869.fastq \
    -o 01_rawReads_bat869_trimmed.fastq \
    --threads 16 \
    --check_reads 50000 \
    --adapter_threshold 85 \
    --end_size 200 \
    --min_split_read_size 200
```

Explaining the parameters for the slurm script: -i the input file -o the name of the output files to be created –threads CPU threads used for parallel processing –check_reads subset of reads aligned to all known adapter sets –adapter_threshold percent identity threshold to trim adapters at read ends –end_size first and last bases in each read checked for adapters –min_split_read_size minimum length of reads to keep after splitting

Narrative questions:

1. What are sequencing adapters and why are they particularly important to remove for Nanopore reads? The sequencing adapters are used for Nanopore sequencing. They are short DNA sequences attached to the DNA fragments so the Nanopore machine can read and sequence them. It's important to remove adapters because they can get sequenced as part of the genome and if they aren't removed they will cause problems when mapping and assembling the genome.

2. Describe which main Porechop parameters you would adjust (e.g., minimum adapter match, read splitting) and why in this dataset. My main goal in adjusting Porechop parameters was to make sure all adapters were being cut. I decreased the adapter threshold by 5% from the default to increase sensitivity to adapters, making it more likely that Porechop trims everything it should. I increased the end size by 50 bp to trim a few extra bases at the ends since that is where the adapters are. I also increased the minimum length of reads to keep after splitting from 100 to 200 bp to remove short reads, improving the quality. I also increased the number of reads checked from 10000 to 50000 so adapter detection would cover more of the dataset at once, which also helped it run a bit faster.

3. What output(s) do you expect from Porechop and how will you verify trimming worked (describe at least one QC check)? Porechop takes a FASTQ file and produces a trimmed FASTQ. After trimming the adapters, I expected shorter reads. As a QC check, I compared the number of reads and read lengths before and after trimming, and looked for any leftover adapter sequences in the trimmed reads. Since the main goal is to remove adapters, confirming that no adapters remain is the most important QC check.

#SLURM script for QC post porechop

```
#!/bin/bash
#SBATCH --job-name=qc_porechop
#SBATCH --output=qc_porechop_%j.out
#SBATCH --error=qc_porechop_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=100G
#SBATCH --time=2:00:00
```

```
module load seqkit
#Check stats
seqkit stat 00_rawReads_bat869.fastq \
            01_rawReads_bat869_trimmed.fastq \
            > qc_porechop_stats.txt
#Count reads
grep -c "^@" 00_rawReads_bat869.fastq >> qc_porechop_stats.txt
grep -c "^@" 01_rawReads_bat869_trimmed.fastq >> qc_porechop_stats.txt
#Check for leftover adapter sequences
grep -c "AAGCAGTGGTATCAACGCAGAGTAC" 01_rawReads_bat869_trimmed.fastq >> qc_porechop_stats.txt
grep -c "TTTCTGTTGGTGCTGATATTGCTG" 01_rawReads_bat869_trimmed.fastq >> qc_porechop_stats.txt
```

Part 4: Quality Filtering with fastplong

#SLURM script for fastplong

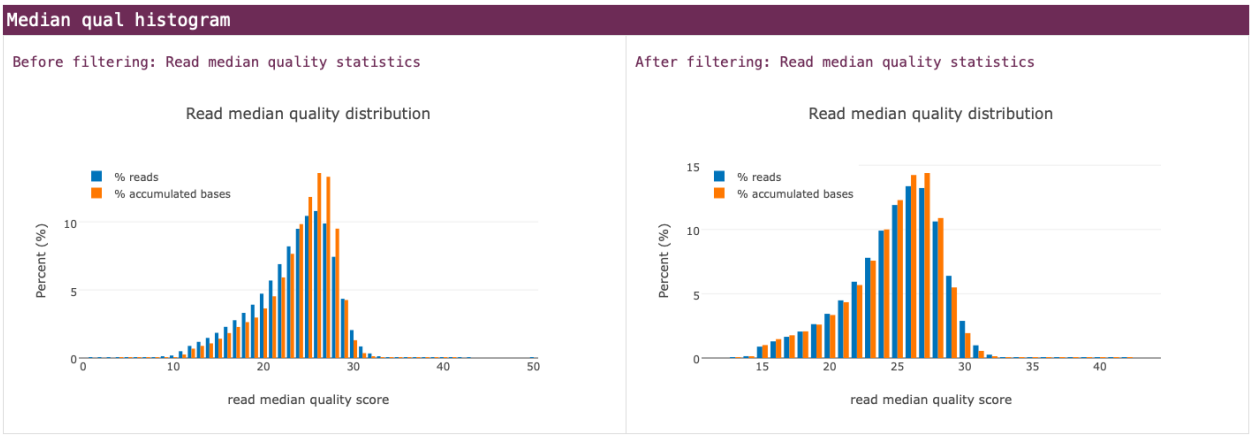
```
#!/bin/bash
#SBATCH --job-name=fastplong
#SBATCH --output=fastplong_%j.out
#SBATCH --error=fastplong_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=100G
#SBATCH --time=6:00:00

    /users/mserkin/fastplong/fastplong.bin \
    -i 01_rawReads_bat869_trimmed.fastq \
    -o 02_bat869_clean.fastq \
    -l 600 \
    -q 12 \
    -t 16 \
    -s TGGTTCAGTT
```

Explaining the parameters for the slurm script: -i the input file -o the name of the output files to be created -l the minimum read length to keep -q the minimum quality to keep -t CPU threads used for parallel processing -s adapter DNA sequence

#Post fastplong QC table or plot

Table 1: Fastplong QC Summary: Before vs After Filtering

Metric	Before	After	Difference
Total Reads	6172144	2223730	-3948414
Total Bases	11215783342	5441113263	-5774670079
Mean Length	6172144	2223730	-3948414
N50	11215783342	5441113263	-5774670079



Narrative questions:

1. Define what you consider a "high-quality" Nanopore read in this context (justify chosen min Q and length). A high quality Nanopore read is one that balances good base quality with retaining enough data for downstream analysis. I chose a minimum quality score of 12 because I tested using 10 and decided that raising it slightly would improve overall read reliability without discarding too many reads. I lowered it from the default of 15 since Nanopore data is more noisy and I did not want to discard too many reads that can be useful. The minimum read length of 600 ensures that very short, uninformative reads are removed. I previously tried 500 bp but decided to raise it slightly to remove a few more low information reads while still keeping most of the data. As demonstrated in the "Median qual histogram" graph, after filtering the read median quality score increased as did the percent of reads and bases.

2. What are the consequences of filtering too stringently vs. too leniently for mitochondrial assembly? The consequences for filtering too leniently is that low quality or short reads can remain in the data, which produced noise that affects accuracy when assembling the genome. Filtering too stringently can remove useful data which can then affect the genome assembly by having fragments, gaps and poor coverage.

#SLURM script for QC post fastplong

```
#!/bin/bash
#SBATCH --job-name=qc_fastplong
#SBATCH --output=qc_fastplong_%j.out
#SBATCH --error=qc_fastplong_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=100G
#SBATCH --time=2:00:00

module load seqkit
#Check  stats
seqkit stat 00_rawReads_bat869.fastq \
            01_rawReads_bat869_trimmed.fastq \
            02_bat869_clean.fastq \
            > qc_fastplong_stats.txt
#Read counts
grep -c "^@" 01_rawReads_bat869_trimmed.fastq >> qc_fastplong_stats.txt
```

```
grep -c "^@" 02_bat869_clean.fastq >> qc_fastplong_stats.txt

#Check for very short reads that should have been removed
seqkit seq -m 200 02_bat869_clean.fastq | grep -c "^@" >> qc_fastplong_stats.txt
```

Part 5: Mitochondrial Genome Assembly

#SLURM script for minimap2

```
#!/bin/bash
#SBATCH --job-name=minimap
#SBATCH --error=minimap_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=36
#SBATCH --mem=500G
#SBATCH --time=2:00:00

#Map reads to related species  genome
sh runMinimap.sh 02_bat869_clean.fastq  MN125184.1_mito.fasta
```

#SLURM script for minimap2

```
#!/bin/bash
#SBATCH --job-name=minimap
#SBATCH --error=minimap_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=36
#SBATCH --mem=100G
#SBATCH --time=2:00:00

#Map reads to related species  genome
#sh runMinimap.sh 02_bat869_clean.fastq  MN125184.1_mito.fasta

module load seqtk
#Extract reads from related genome mapping with at least 500bp alignment length to the using Seqtk
awk -F"\t" '($4-$3)>500 {print $1}' 02_bat869_clean_MN125184.1_mito_minimap2.paf \
    | sort \
    | uniq \
    | seqtk subseq 02_bat869_clean.fastq - > Bat_Mitogenome.fastq

seqtk seq -a Bat_Mitogenome.fastq > Bat_Mitogenome.fasta
```

#SLURM script for FLYE

```
#!/bin/bash
#SBATCH --job-name=flye
#SBATCH --error=flye_%j.err
```

```
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=36
#SBATCH --mem=100G
#SBATCH --time=2:00:00

module load flye
module load seqtk

reads="Bat_Mitogenome.fastq"
genome_size=17000
threads=36
output_dir="flye_runs"

# Define 12 different min-overlap values for Flye
min_overlaps=(1000 1100 1200 1300 1400 1500 1600 1700 1800 1900 2000 2500)

# Loop over each min-overlap to create 12 assemblies
mkdir -p $output_dir

for mo in "${min_overlaps[@]}"; do
    outdir="$output_dir/mo${mo}"
    flye --nano-raw "$reads" \
         --threads $threads \
         --out-dir "$outdir" \
         --genome-size $genome_size \
         --min-overlap $mo \
         --iterations 3
done
```

Explaining the parameters for the slurm script: –nano-raw input rae Nanopore reads –threads CPU threads used for parallel processing –out-dir directory for all output files –genome-size expected genome size –min-overlap minimum overlap length between reads –iterations number of polishing cycles

#SLURM script for best FLYE assembly

```
#!/bin/bash
#SBATCH --job-name=flye_best
#SBATCH --error=flye_best_%j.err
#SBATCH --output=flye_best_%j.out
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=10G
#SBATCH --time=00:30:00

module load seqkit

# Print header for the SLURM output
echo -e "Assembly\tContigs\tTotalLength\tCircular"

# Initialize array to store info for best selection
```

```
assembly_dirs=()
contigs_list=()
length_list=()
circular_list=()

# Loop over Flye output folders
for d in flye_runs/mo*; do
    info="$d/assembly_info.txt"
    if [ -f "$info" ]; then
        contigs=$(awk 'NR>1{count++}END{print count}' "$info")
        totlen=$(awk 'NR>1{sum+=$2}END{print sum}' "$info")
        circular=$(awk 'NR>1 && tolower($4)=="y"{c=1}END{if(c==1) print "TRUE"; else print "FALSE"}' "$

        echo -e "$d\t$contigs\t$totlen\t$circular"

        # Store info for later
        assembly_dirs+=("$d")
        contigs_list+=("$contigs")
        length_list+=("$totlen")
        circular_list+=("$circular")
    fi
done

# Select best assembly: 1 contig, circular, closest to 17000 bp
best=""
min_diff=1000000
for i in "${!assembly_dirs[@]}"; do
    if [ "${contigs_list[$i]}" -eq 1 ] && [ "${circular_list[$i]}" == "TRUE" ]; then
        diff=$(( ${length_list[$i]} - 17000 ))
        diff=${diff#-}  # absolute value
        if [ "$diff" -lt "$min_diff" ]; then
            min_diff=$diff
            best="${assembly_dirs[$i]}"
        fi
    fi
done

# If no perfect assembly, pick closest by length
if [ -z "$best" ]; then
    echo "No 1-contig circular assembly found, picking closest length to 17000"
    min_diff=1000000
    for i in "${!assembly_dirs[@]}"; do
        diff=$(( ${length_list[$i]} - 17000 ))
        diff=${diff#-}  # absolute value
        if [ "$diff" -lt "$min_diff" ]; then
            min_diff=$diff
            best="${assembly_dirs[$i]}"
        fi
    done
fi

echo -e "\nBest assembly selected: $best"
```

```
# Copy best assembly
best_fasta="$best/assembly.fasta"
if [ -f "$best_fasta" ]; then
    cp "$best_fasta" "Bat_Best_Mitogenome.fasta"
    echo "Best assembly copied to Bat_Best_Mitogenome.fasta"
else
    echo "Error: assembly.fasta not found in $best"
fi
```

#SLURM script for polishing using Racon

```
#!/bin/bash
#SBATCH --job-name=racon
#SBATCH --output=racon_%j.out
#SBATCH --error=racon_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=36
#SBATCH --mem=100G
#SBATCH --time=4:00:00

module load minimap2
module load racon

# Input files
reads="Bat_Mitogenome.fastq"
assembly="Bat_Best_Mitogenome.fasta"

# Number of threads
threads=36

# Number of polishing rounds
rounds=3
current_assembly=$assembly

for i in $(seq 1 $rounds); do
    echo "=== Polishing round $i ==="

    # Step 1: Map reads to current assembly
    minimap2 -t $threads -ax map-ont $current_assembly $reads > racon_round${i}.sam

    # Step 2: Run Racon to polish
    polished="Bat_Best_Mitogenome_polished_round${i}.fasta"
    racon -t $threads $reads racon_round${i}.sam $current_assembly > $polished

    # Step 3: Prepare for next round
    current_assembly=$polished

    echo "Round $i complete. Output: $current_assembly"
done

# Final polished assembly
final_assembly="Bat_Best_Mitogenome_polished.fasta"
```

```
cp $current_assembly $final_assembly
echo "Final polished assembly: $final_assembly"
```

Narrative questions:

1. Explain how you adapted an organellar genome assembly workflow for animal mitochondrial DNA (no chloroplast step, considerations for circular genomes). I adapted an organellar genome assembly workflow for animal mitochondrial DNA Since animal mitochondrial genomes are circular, I considered this in selecting assemblies that were single contig and circular. I filtered out NUMTs by mapping the reads to a reference mitochondrial genome, which allowed me to keep only reads that aligned properly and reduce nuclear contamination. I polished my Flye assembly using Racon, following the polishing step in the workflow. While the original tutorial used multiple assemblers, I only used Flye because other tools were not available on the cluster.
2. Describe at least two likely challenges in assembling this mitochondrial genome from Nanopore data (e.g., homopolymers, NUMTs, low coverage) and how you can design a pipeline addresses them. One challenge is repetitive regions (like homopolymers) that can cause errors in Nanopore reads. I ran Racon three times to polish and make a final consensus, which improved accuracy in those regions. Another challenge is low coverage. I first mapped reads to a reference genome and only kept reads that matched well, then ran Flye with different minimum-overlap settings to make several assemblies. Picking the best assembly helped make a stronger final genome.
3. Briefly justify your choice of assembler or reference-guided approach in 1–2 paragraphs. I used Flye because it works well with noisy Nanopore reads, circular genomes, and repeats, and it was easy to use. Using Minimap2 to map to a reference helped pick only the reads relevant to the mitochondrial genome and remove NUMTs. I also polished the final Flye assembly with Racon to make the genome more accurate.

Part 6: Annotation and Assembly Quality Metrics

#SLURM script for MitoZ

```
#!/bin/bash
#SBATCH --job-name=mitoz
#SBATCH --output=mitoz_%j.out
#SBATCH --error=mitoz_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=100G
#SBATCH --time=6:00:00

# Paths
WORKDIR=/users/mserkin/mito_genome/Bat_mito_final
INPUT_FNA=$WORKDIR/Bat_Best_Mitogenome_polished.fasta
CIRCULAR_FNA=$WORKDIR/Bat_Best_Mitogenome_polished_circular.fasta
OUTPUT_DIR=$WORKDIR/MITOSZ_Bat

# Create output folder if it doesn't exist
mkdir -p $OUTPUT_DIR
```

```
# Make a circular copy of the polished assembly
cp $INPUT_FNA $CIRCULAR_FNA
sed -i 's/^>/>circular /' $CIRCULAR_FNA

# Run MitoZ annotation
singularity run --no-mount /scratch --no-mount /vast ~/MitoZ_3.6.sif mitoz annotate \
    --fastafiles $CIRCULAR_FNA \
    --outprefix Bat_Best_Mitogenome \
    --workdir $OUTPUT_DIR \
    --thread_number 16 \
    --clade Chordata \
    --genetic_code 2
```

Explaining the parameters for the slurm script: -fastafiles input fasta file –outprefix prefix given to all output files –workdir directory where ouput files are written –thread_number CPU threads used for parallel processing –clade taxonomic group of dataset organism -genetic_code translation table code

#Annotation Genes output # A tibble: 74 x 5 gene start end strand product

1 CYTB 1 819 +
2 CYTB 1 819 + cytochrome b
3 trnT(ugu) 819 888 + tRNA-Thr
4 trnT(ugu) 819 888 + tRNA-Thr
5 trnP(ugg) 888 953 - tRNA-Pro
6 trnP(ugg) 888 953 - tRNA-Pro
7 trnF(gaa) 2195 2264 + tRNA-Phe
8 trnF(gaa) 2195 2264 + tRNA-Phe
9 s-rRNA 2265 3228 + 12S ribosomal RNA 10 s-rRNA 2265 3228 + 12S ribosomal RNA # i 64 more rows

#Summary of mitochondrial genes annotated by MitoZ

```
# Path to MitoZ summary file
summary_file <- "summary.txt"

# Read file
lines <- readLines(summary_file)

# Extract numbers
protein_coding <- as.numeric(str_extract(
  lines[str_detect(lines, "Protein coding genes")], "\\d+"
))

trna <- as.numeric(str_extract(
  lines[str_detect(lines, "tRNA genes")], "\\d+"
))

rrna <- as.numeric(str_extract(
  lines[str_detect(lines, "rRNA genes")], "\\d+"
))

# Create summary table
gene_summary <- tibble(
  Category = c("Protein-coding genes", "tRNA genes", "rRNA genes", "Total genes"),
  Count = c(
```

```
    protein_coding,
    trna,
    rrna,
    protein_coding + trna + rrna
  )
)

kable(
  gene_summary,
  caption = "Summary of mitochondrial genes annotated by MitoZ"
)
```

Table 2: Summary of mitochondrial genes annotated by MitoZ

Category	Count
Protein-coding genes	13
tRNA genes	22
rRNA genes	2
Total genes	37

#Mitochondrial genome assembly metrics

```
# Assembly metrics
fasta <- readDNAStringSet("Bat_Best_Mitogenome_polished.fasta")

assembly_length <- sum(width(fasta))
gc_content <- sum(letterFrequency(fasta, c("G", "C"))) / assembly_length * 100
num_contigs <- length(fasta)

metrics <- tibble(
  Metric = c("Total length (bp)", "GC content (%)", "Number of contigs", "Circular"),
  Value = c(
    assembly_length,
    round(gc_content, 2),
    num_contigs,
    "Yes"
  )
)

kable(metrics, caption = "Mitochondrial genome assembly metrics")
```

Table 3: Mitochondrial genome assembly metrics

Metric	Value
Total length (bp)	16696
GC content (%)	41.84
Number of contigs	1
Circular	Yes

Narrative questions:

1. Which canonical mitochondrial genes do you expect for a mammalian (bat) mitochondrion, and which did you actually detect? A mammalian(bat) mitochondrion has 37 genes: 13 protein coding, 2 ribosomal RNA genes, and 22 transfer RNA genes. I also detected 37 coding genes and as seen in the "Summary of mitochondrial genes annotated by MitoZ" table, I also detected 13 protein coding, 2 ribosomal RNA genes, and 22 transfer RNA genes. All 37 genes can be viewed on the "gene_table".

2. How do your assembly length and GC content compare to typical bat mitochondrial genomes? Provide some citations. The brandt's bat(Myotis brandtii) is found to be 17,470 bp in length and GC content of 38.2%(https://pubmed.ncbi.nlm.nih.gov/25103443/). The large flying fox (Pteropus vampyrus) is 16,554 bp in length and has a GC content of 41.5%(https://pubmed.ncbi.nlm.nih.gov/25600745/). The lesser short-nosed fruit bat (Cynopterus brachyotis) has 16,701 bp and a GC content of 41.9%(https://pubmed.ncbi.nlm.nih.gov/25418628/). The assembly length of my bat genome is 16696 bp with a GC content of 41.84%. Based on the length and GC content, my genome assembly is most similar to the lesser short-nosed fruit bat.

3. Based on your metrics and gene content, how confident are you that this assembly represents a complete, high-quality mitogenome? I am very confident that my assembly represents a complete, high quality mitogenome. My mitogenome is circular, has 1 contig, has a reasonable length and GC content, and detecting the correct number of genes and types of genes. All of these variables point to a complete, high quality mitogenome.

Part 7: Species Identification

#SLURM script for BLAST

```
#!/bin/bash
#SBATCH --job-name=blast_mito
#SBATCH --output=blast_mito_%j.out
#SBATCH --error=blast_mito_%j.err
#SBATCH --partition=Centaurus
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=1:00:00

# Load BLAST module
module load blast/2.15.0+

# Query: your assembled mitochondrial genome
QUERY="/users/mserkin/mito_genome/Bat_mito_final/Bat_Best_Mitogenome_polished.fasta"

# Output file
OUTFILE="BatMito_blast.out"

# Run BLASTn remotely against NCBI nt database
blastn -query $QUERY \
       -db nt \
       -remote \
       -out $OUTFILE \
       -outfmt "6 qseqid sseqid pident length mismatch gapopen evalue bitscore stitle" \
       -max_target_seqs 10
```

Explaining the parameters for the slurm script: -query input fasta file -db nt database to search- NCBI nucleotide database -remote runs BLAST on NCBI's servers -out output file -outfmt ouput tabular format -max_target_seqs returns 10 top hits

#BLAST Results

```
blast <- read_tsv(
  "blast_results.out",
  col_names = c(
    "query",
    "subject",
    "percent_identity",
    "alignment_length",
    "mismatches",
    "gap_opens",
    "evalue",
    "bitscore",
    "species"
  )
)
```

```
## Rows: 27 Columns: 9
## -- Column specification --------------------------------------------------------
## Delimiter: "\t"
## chr (3): query, subject, species
## dbl (6): percent_identity, alignment_length, mismatches, gap_opens, evalue, ...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
blast
```

```
## # A tibble: 27 x 9
##    query   subject percent_identity alignment_length mismatches gap_opens evalue
##    <chr>   <chr>              <dbl>            <dbl>      <dbl>     <dbl>  <dbl>
##  1 contig~ gi|182~             99.4            14506         84         5      0
##  2 contig~ gi|182~             98.0             1874         37         0      0
##  3 contig~ gi|182~             98.2              508          6         2      0
##  4 contig~ gi|182~             99.3            14506         89         5      0
##  5 contig~ gi|182~             97.5             2198         49         4      0
##  6 contig~ gi|182~             93.0            14509        999        11      0
##  7 contig~ gi|182~             91.5             2228        147        20      0
##  8 contig~ gi|182~             93.0            14510        998        13      0
##  9 contig~ gi|182~             93.2             1898        113        13      0
## 10 contig~ gi|182~             94.1              522         25         3      0
## # i 17 more rows
## # i 2 more variables: bitscore <dbl>, species <chr>
```

#BLAST Species Stats

```
blast <- blast %>%
  mutate(
    species_clean = str_extract(species, "Rousettus\\s+[a-z]+")
```

```
  )

species_summary <- blast %>%
  group_by(species_clean) %>%
  summarise(
    hits = n(),
    mean_identity = round(mean(percent_identity), 2),
    max_bitscore = max(bitscore),
    max_alignment = max(alignment_length)
  ) %>%
  arrange(desc(max_bitscore))

knitr::kable(
  species_summary,
  caption = "BLAST species summary"
)
```

Table 4: BLAST species summary

species_clean	hits	mean_identity	max_bitscore	max_alignment
Rousettus leschenaultii	5	98.50	26284	14506
Rousettus aegyptiacus	19	92.80	21185	14511
Rousettus obliviosus	3	92.93	21126	14513

Narrative questions:

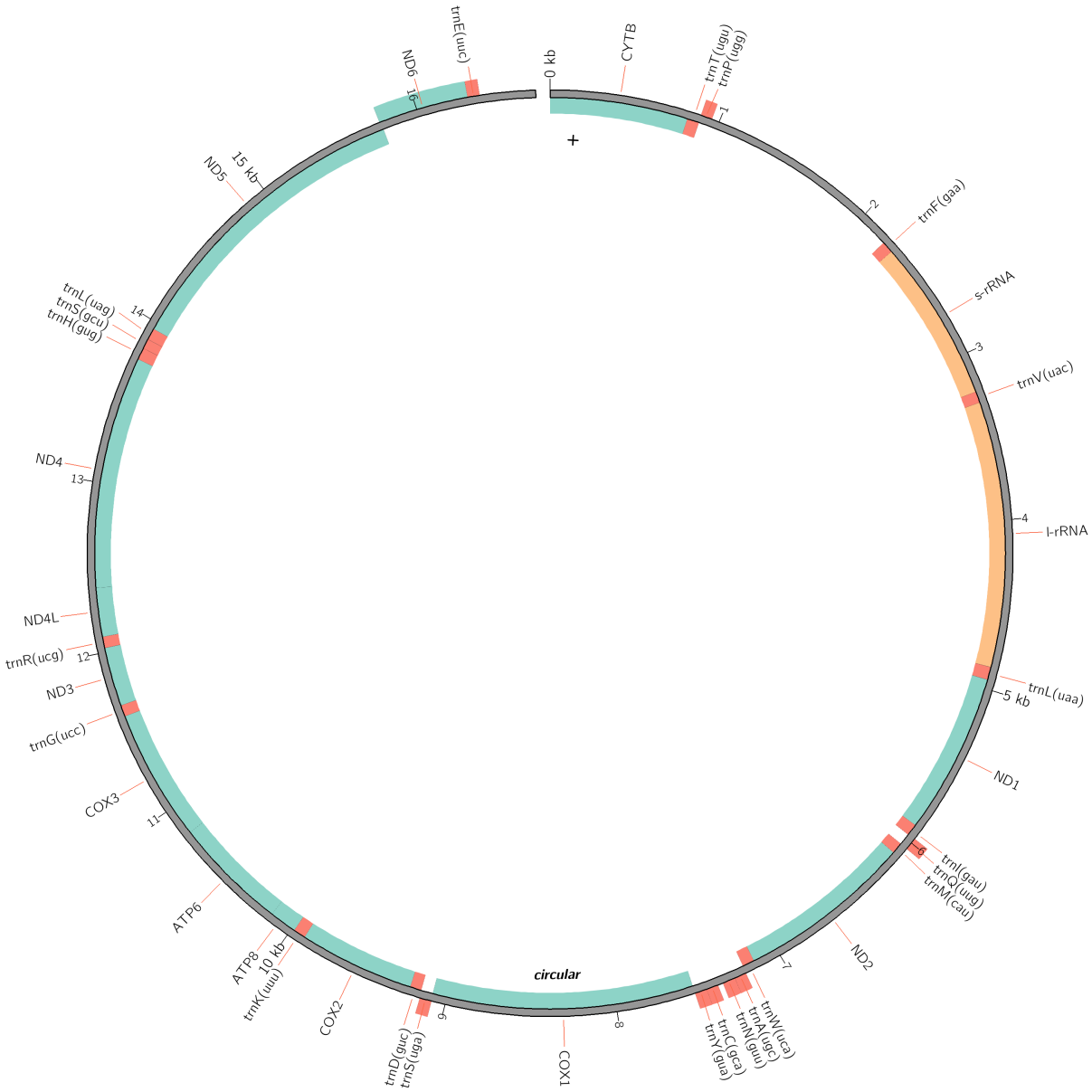
1. Describe your species identification strategy in 1–2 paragraphs, including the database(s) you would use and why. For species identification, I used my final, polished fasta file and used BLAST to run it against the NCBI nucleotide database. Since the assembled genome is quite small, I wrote a slurm script to run the BLAST in the cluster. I used the NCBI nucleotide database because it is frequently updated and it contains the mitochondrial genomes. I am confident that this will provide me with the closest matching species, helping me identify our mystery bat.

2. Explain how you would interpret similarity/hit scores to distinguish between closely related bat species. The higher the similarity score, the more likely its the correct bat species. The correct species typically will not only have the highest mean identity score, but also highest bitscore and best alignment. I also look if the top hits come back to the same species or if the top hits varied between species.

Part 8: Discussion

#Mitochondrial Genome Assembly

```
knitr::include_graphics("circos.png")
```



1. Summarizes the main findings (quality of assembly, completeness, species ID). Starting with the raw Nanopore reads, I trimmed adapters, filtered out low-quality and short reads, and assembled the mitogenome. The final assembly is circular, 16,696 bp long, with a GC content of 41.84%, and includes all 37 canonical mitochondrial genes. The assembled genome is seen in the image above. Based on the "BLAST species summary" plot, Rousettus aegyptiacus had the highest number of hits (19), but Rousettus leschenaultii had a higher mean identity score (98.50%), the highest bit score (26,284), and the longest maximum alignment (14,506 bp). These metrics provide strong evidence that the assembly belongs to Rousettus leschenaultii.

2. Reflects on limitations and possible improvements to the pipeline. Nanopore reads can be prone to NUMTs, repeats, and low coverage. I addressed these challenges, but I was limited by the tools available on the cluster and could not use some of the most accurate programs. To improve the pipeline, I would try to get MitoZ working more accurately, at some points it did not mark the genome as circular and produced a different species identification than BLAST. Using an additional annotation tool could help

confirm the results. For polishing, Medaka could be used after Racon to further polish the Nanopore reads. In general, streamlining the code and workflow could increase efficiency and reproducibility.